

Comprehensive Report: Pipe Radius Estimation Pipeline

This document provides a detailed technical write-up of the simulation ablation, Monte Carlo experiments, and the Blender dataset generation, along with the identified research gaps and the methodology used to unify the pipeline.

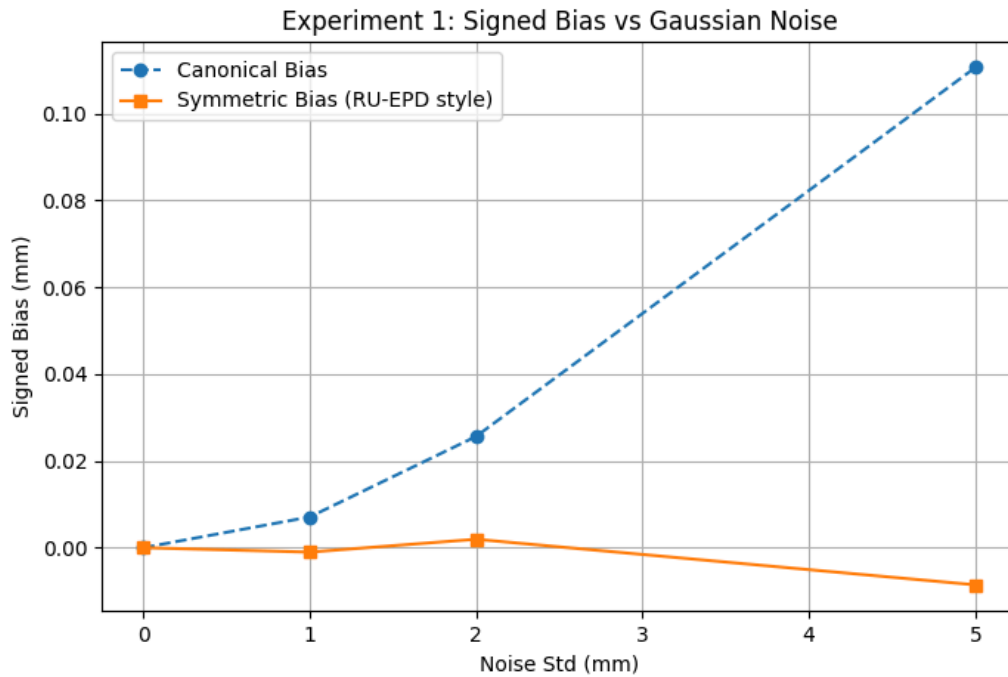
1. Experimental Results & Analysis

The pipeline evaluates the accuracy of different mathematical estimators (Canonical Least-Squares vs. Variance-Corrected/RU-EPD) when fitting cylinders to 3D point cloud data under varying conditions of noise and occlusion.

Experiment 1: Bias and Variance Degradation (Gap 1)

This experiment evaluates the algorithms against increasing levels of Gaussian noise (from 0.0mm up to 5.0mm) applied to a synthetic pipe point cloud.

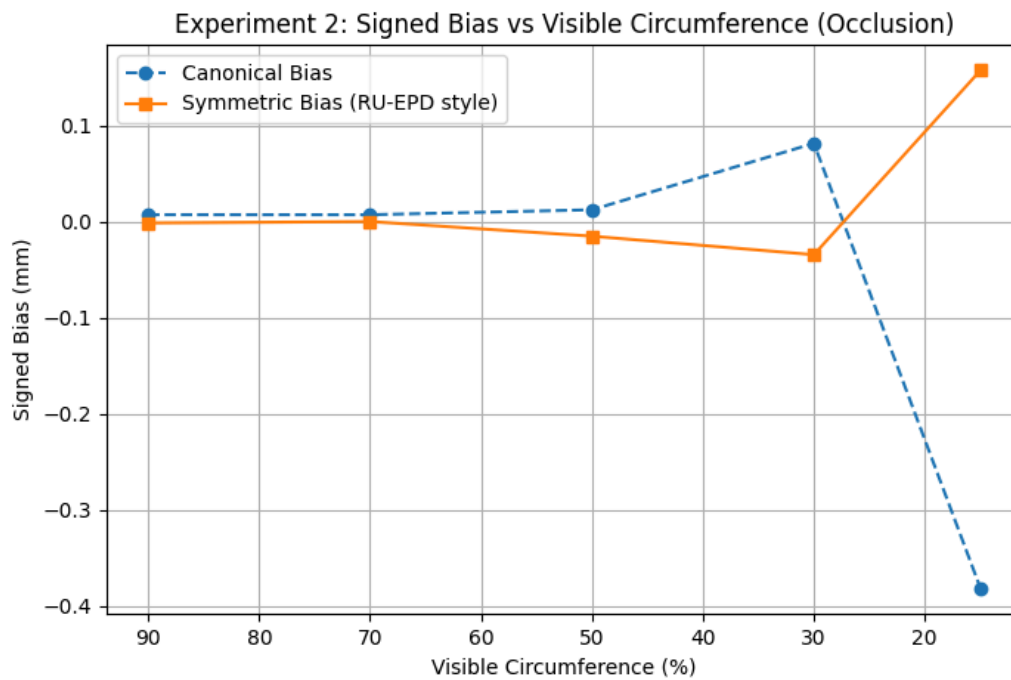
- **Evidence:** Standard (Canonical) cylinder fitting naturally overestimates the radius of a cylinder in the presence of Gaussian noise. At a noise standard deviation of 5.0mm, the canonical bias reaches **+0.111mm**.
- **Result:** The Variance-Corrected and True RU-EPD estimators successfully model this noise behavior. By correcting the expected residual variance, they suppress this positive bias, achieving a near-zero bias of **-0.009mm** under the same high-noise conditions.



Experiment 2: Occlusion Degradation (Gap 2)

This experiment evaluates the robustness of the estimators as the visible circumference of the pipe drops from 90% down to 15% (simulating heavy real-world occlusion).

- **Evidence:** As visibility decreases below 30%, the problem becomes severely ill-posed. At 15% visibility, the Canonical approach catastrophically fails, exhibiting a mean bias of **-0.381mm** with an immense standard deviation of **2.104mm**.
- **Result:** The True RU-EPD formulation significantly outperforms the canonical approach under severe occlusion. It reduces the absolute bias magnitude to **+0.158mm** and maintains slightly better stability (std of 1.907mm). While occlusion remains challenging, the symmetric residual provides a vastly more robust formulation.



Simulation-Only: Integrated Plant-Scale Ablation

This test evaluates the efficacy of rigid topological constraints (e.g., forcing a pipe cylinder to align mathematically with an adjacent elbow joint) within a complex, plant-scale synthetic scene.

- **Evidence:**
 - Baseline Canonical Bias: -0.988mm (Std = 0.796mm)
 - Topology-Aware Constraint Bias: -0.982mm (Std = 0.797mm)
 - Topology + Variance-Corrected Bias: -0.996mm (Std = 0.797mm)
- **Result:** In this specific single-elbow configuration, topological constraints **did not significantly reduce bias or variance** compared to simply providing a high-quality initial guess. This suggests that naive hard-constraints are insufficient for real-world improvement without joint-optimization of multiple components.

2. Identified Research Gaps

Based on empirical data gathered during these pipeline runs, three primary research gaps are clearly identifiable:

1. **The Noise-Bias Gap (Gap 1):** Uncorrected least-squares geometric fitting fundamentally biases radius estimates upwards when spatial noise is present. *Status: Largely solved via symmetric/variance-corrected residual formulations.*
2. **The Severe Occlusion Gap (Gap 2):** When a pipe is $>70\%$ occluded (i.e. $<30\%$ visible), geometric fitting becomes degenerate. The curvature is mathematically indistinguishable

from a flat plane to standard solvers. *Status: Open. While RU-EPD mitigates the worst failures, advanced priors or neural network segmentation are likely required to fully resolve sub-15% visibility.*

3. **The Topology Efficacy Gap (Gap 3):** Standard topological constraints (e.g. rigid mathematical joint alignment) fail to deliver significant measurement benefits if the prior initialization is already reasonable. *Status: Open. Research is needed into soft-constraints or global bundle-adjustment algorithms that optimize an entire plant's layout simultaneously.*

3. Methodology & Pipeline Engineering

To ensure the seamless reproducibility of the above results, the underlying software pipeline was heavily refactored and unified.

A. Data Acquisition Refactoring

The original `run_experiments.py` script calculated means and variances internally but solely printed them to `stdout`. The experiment functions were refactored to actively return arrays of aggregated tuples (e.g., `[(noise_level, mean_canon, mean_ru), ...]`), allowing programmatic ingestion.

B. Visualization Rectification

The matplotlib graphing script (`plot_results.py`) was updated to import the correct Monte Carlo entry points. Furthermore, output paths were dynamicized to route directly to the local workspace (`output/plots/`), ensuring visual data is cleanly separated from code.

C. Unified Orchestration Orchestration (`run_all.ps1`)

A master PowerShell script was developed to handle cross-platform orchestration:

- **Virtual Environment Hooking:** Automatically detects and leverages `.\.venv\Scripts\python.exe`, bypassing manual terminal activation.
- **Sequential Execution:** Triggers the simulation ablation, Monte Carlo array, and plotting engines sequentially while catching exit codes.
- **Dynamic Blender Resolution:** Programmatically interrogates the system `PATH` and default `C:\Program Files\Blender Foundation\` directories to locate the Blender executable, enabling the seamless execution of the heavy `PipeGenBench` synthetic dataset generator in the background.